

学号：202300130045	姓名：张博文	班级： 数据 23
实验题目： bert 实验		
实验学时： 2	实验日期： 2025/11/17	
实验目的：熟悉 PyTorch 框架下，利用预训练的 transformers 的预训练 BERT 模型对 MRPC 数据集进行同义预测的 pipeline. 尝试理解数据是如何预处理，模型是怎么读入数据，是如何进行推理，如何进行评价的.		
硬件环境： 计算机一台		
软件环境： Linux，AutoDL 算力云		
实验步骤与内容： 本实验目标是利用预训练的 BERT 模型对 MRPC 数据集进行微调，实现句子对的同义预测（二分类）。 一、 环境配置与初始化 (Setup) 1. 环境准备： 连接远程 GPU 服务器，配置 Anaconda 虚拟环境，并安装 PyTorch、HuggingFace Transformers 和 Datasets 等核心库。 2. 基础设置： 在代码开始处设置随机种子（如 42），确保实验的可复现性。 二、 数据加载与预处理 (Data Processing) 3. 加载数据与分词器： 使用 datasets 库加载 GLUE 基准测试集中的 mrpc 子集。同时，加载 bert-base-uncased 对应的分词器（Tokenizer）。 4. 数据映射： 使用分词器对数据集中的句子对（sentence1 和 sentence2）进行批量分词、添加特殊标记（如[CLS]、[SEP]）和截断处理。 5. 创建 DataLoader： 实例化 DataCollatorWithPadding 实现动态填充(Dynamic Padding)，并创建用于训练和评估的 PyTorch DataLoader。 三、 模型训练与评估 (Training & Evaluation) 6. 模型加载与配置： 加载 AutoModelForSequenceClassification（num_labels=2）预训练模型。配置 AdamW 优化器和学习率调度器。 7. 训练循环： 在设定的 Epochs 内进行迭代训练。在每个批次中，执行前向传播、计算损失、反向传播和梯度更新。 8. 模型评估： 在每轮训练结束后，将模型切换至评估模式（model.eval()），在验证集上进行推理，并计算准确率（Accuracy）和F1 分数等指标。 运行结果如图：		

```

Epoch 1/3
训练: 100% | 230/230 [00:09:00:00, 24.03it/s, loss=0.311]
训练损失: 0.5412
评估: 100% | 26/26 [00:00:00:00, 89.09it/s]
验证集准确率: 0.8015, F1分数: 0.8457

Epoch 2/3
训练: 100% | 230/230 [00:09:00:00, 25.12it/s, loss=0.0861]
训练损失: 0.3306
评估: 100% | 26/26 [00:00:00:00, 90.58it/s]
验证集准确率: 0.8480, F1分数: 0.8885

Epoch 3/3
训练: 100% | 230/230 [00:09:00:00, 25.08it/s, loss=0.0177]
训练损失: 0.1971
评估: 100% | 26/26 [00:00:00:00, 89.32it/s]
验证集准确率: 0.8407, F1分数: 0.8866

在测试集上最终评估...
评估: 100% | 26/26 [00:00:00:00, 89.84it/s]
测试集准确率: 0.8407
测试集 F1分数: 0.8866
模型已保存到 ./bert_mrpc_finetuned
(bert-mrpc) root@autodl-container-1b2041a5c3-1e506ba2:~#

```

实验代码如下：

```

import torch
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    DataCollatorWithPadding,
    get_scheduler
)
from datasets import load_dataset
from torch.utils.data import DataLoader
from tqdm.auto import tqdm
import numpy as np
from sklearn.metrics import accuracy_score, f1_score
import random

# --- 1. 初始化设置 ---

# 设置随机种子以保证实验可复现性
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

# 定义超参数和全局变量
MODEL_NAME = "bert-base-uncased"
BATCH_SIZE = 32
EPOCHS = 3
LEARNING_RATE = 2e-5
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# --- 2. 数据加载与预处理 ---

```

2.1 加载 MRPC 数据集

```
print("加载 MRPC 数据集...")
raw_datasets = load_dataset("glue", "mrpc")
```

2.2 加载分词器

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

2.3 定义分词函数

```
def tokenize_function(examples):
    # 对句子对进行分词, 自动添加 [CLS] 和 [SEP]
    return tokenizer(
        examples["sentence1"],
        examples["sentence2"],
        truncation=True # 自动截断超长序列
    )
```

2.4 应用分词 (移除未使用的列, 保留 labels)

```
print("开始分词...")
tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)
tokenized_datasets = tokenized_datasets.remove_columns(["sentence1", "sentence2", "idx"])
```

2.5 创建数据整理器

```
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

2.6 创建 DataLoader

```
train_dataloader = DataLoader(
    tokenized_datasets["train"],
    shuffle=True,
    batch_size=BATCH_SIZE,
    collate_fn=data_collator
)
eval_dataloader = DataLoader(
    tokenized_datasets["validation"],
    batch_size=BATCH_SIZE,
    collate_fn=data_collator
)
test_dataloader = DataLoader(
    tokenized_datasets["test"],
    batch_size=BATCH_SIZE,
    collate_fn=data_collator
)
```

--- 3. 模型与优化器初始化 ---

```

# 3.1 加载预训练模型 (num_labels=2 for binary classification)
print(f'加载模型 {MODEL_NAME}...')
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)
model.to(DEVICE)

# 3.2 配置优化器
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

# 3.3 设置学习率调度器
num_training_steps = len(train_dataloader) * EPOCHS
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps
)

# --- 4. 训练与评估函数 ---

# 4.1 训练函数
def train():
    model.train()
    total_loss = 0
    progress_bar = tqdm(train_dataloader, desc="训练")

    for batch in progress_bar:
        # 将数据移到指定设备 (如 GPU)
        batch = {k: v.to(DEVICE) for k, v in batch.items()}

        outputs = model(**batch)
        loss = outputs.loss

        # 反向传播与优化
        loss.backward()
        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()

        total_loss += loss.item()
        progress_bar.set_postfix({'loss': loss.item()})

    return total_loss / len(train_dataloader)

# 4.2 评估函数

```

```

def evaluate(dataloader, desc="评估"):
    model.eval()
    predictions = []
    true_labels = []

    progress_bar = tqdm(dataloader, desc=desc)

    with torch.no_grad():
        for batch in progress_bar:
            batch = {k: v.to(DEVICE) for k, v in batch.items()}

            outputs = model(**batch)
            logits = outputs.logits

            # 获取预测结果
            preds = torch.argmax(logits, dim=1)

            predictions.extend(preds.cpu().numpy())
            true_labels.extend(batch['labels'].cpu().numpy())

    # 计算评估指标
    accuracy = accuracy_score(true_labels, predictions)
    f1 = f1_score(true_labels, predictions)

    return accuracy, f1, predictions, true_labels

# --- 5. 主执行逻辑 ---

print("\n 开始训练....")
for epoch in range(EPOCHS):
    print(f"\n===== Epoch {epoch + 1}/{EPOCHS} =====")

    # 训练
    train_loss = train()
    print(f"训练损失: {train_loss:.4f}")

    # 验证
    val_accuracy, val_f1, _, _ = evaluate(eval_dataloader, desc="验证")
    print(f"验证集准确率: {val_accuracy:.4f}, F1 分数: {val_f1:.4f}")

# 最终测试
print("\n 在测试集上最终评估...")
test_accuracy, test_f1, test_predictions, test_true_labels = evaluate(test_dataloader, desc="测试")
print(f"测试集准确率: {test_accuracy:.4f}")

```

```
print(f"测试集 F1 分数: {test_f1:.4f}")

# 保存模型
model.save_pretrained("./bert_mrpc_finetuned")
tokenizer.save_pretrained("./bert_mrpc_finetuned")
print("\n 模型已保存至 ./bert_mrpc_finetuned 目录。")
```

结论分析与体会：

本实验成功实现了基于预训练 **BERT 模型**对 MRPC 数据集的微调，用于判断句子对的同义性。BERT 通过其强大的**语义理解能力**，在句子对分类任务上取得了较高的**准确率和 F1 分数**。体会到使用 transformers 库能够极大地简化深度学习模型的部署与微调流程，证明了**预训练模型**在特定 NLP 任务上迁移学习的高效性。